

A Comparative Analysis of Models for Real-Time Personal Protective Equipment Detection on Edge Devices

Giustino Claudio Miglionico, Francesco Di Rienzo, Pietro Ducange, Francesco Marcelloni, Carlo Vallati
Department of Information Engineering, University of Pisa, Largo Lucio Lazzarino 1, 56122 Pisa, Italy
Email: giustino.miglionico@phd.unipi.it, francesco.dirienzo@ing.unipi.it
{pietro.ducange, francesco.marcelloni, carlo.vallati}@unipi.it

Abstract—The use of personal protective equipment (PPE) is essential to improve workplace safety. Despite specific regulations requiring the use of PPE, workers often neglect to wear it due to factors such as inattention, urgency, or convenience. Monitoring the correct use of PPE is especially critical in high-risk tasks. Computer vision technology can automate this process, leveraging deep neural models. This study investigates the performance of modern object detection models in identifying the correct use of PPE, focusing on their accuracy and execution speed. Specifically, the YOLOv11 and RT-DETR models are employed, trained on a real-world PPE dataset. Deployment on low-cost hardware, specifically an NVIDIA Jetson Nano, is evaluated using three deployment frameworks, namely PyTorch, OpenVINO, and TensorRT. The results show that YOLOv11n, with 2.6 million parameters, provides slightly lower average accuracy than more complex models. It stands out for its speed, reaching performances of 6.6 Frames Per Second (FPS) with PyTorch, 2.3 FPS with OpenVINO, and 10.6 FPS with TensorRT. On the other hand, YOLOv11l and YOLOv11x with, respectively, 46.5 and 86.7 million parameters offer higher accuracy, especially evident in small class identification, where simpler models tend to struggle. However, they show lower throughput, with 1.2 and 0.7 FPS on PyTorch. RT-DETR has competitive accuracy but lower performance on edge devices.

Index Terms—Object Detection, Personal Protective Equipment, Edge Devices, Industrial Safety.

I. INTRODUCTION

Workplace safety is the highest priority, especially in industrial settings, where compliance with personal protective equipment (PPE) regulations is essential to prevent accidents. Working conditions can vary widely in terms of risk to worker safety [1]. In low-risk environments, operating restrictions are generally less stringent, while in high-risk settings it is essential to implement stringent safety protocols to protect the safety of operators.

Traditional PPE provides passive protection but lacks the ability to adapt to changes in environmental and operational conditions dynamically. In contrast, smart PPE represents a technological advancement through the integration of advanced sensors, IoT connectivity, and artificial intelligence

(AI) algorithms [2]. These innovative tools enable real-time monitoring of workers' physical parameters and surrounding environmental conditions, delivering proactive alerts that help prevent hazardous situations. Some types of smart PPE include: helmets that use sensors such as accelerometers and gyroscopes to detect falls; vests that integrate biometric sensors to detect critical health conditions or cooling systems; and shoes and belts, equipped with pressure sensors and inertial units to monitor posture and movement.

Regardless of the type, it is, however, crucial to verify whether PPE is being worn appropriately. Monitoring performed by human operators has significant limitations, such as the use of time and resources and inaccuracies due to fatigue or distraction. Advanced technological systems, such as RFID (Radio-Frequency Identification) based monitoring [3], enable verification of PPE usage at the entrances of hazardous areas; however, they cannot ensure that PPE is being worn correctly.

The evolution of deep learning has significantly improved the performance of object recognition systems in computer vision, progressing through three distinct phases. Initially, two-stage models [4] were proposed, demonstrating the ability to achieve high accuracy but at the cost of slow processing times. Subsequently, the advent of single-stage models [5], such as YOLO (You Only Look Once) [6], revolutionized the field by significantly enhancing speed and further accuracy. Today, YOLO is regarded as one of the state-of-the-art object detectors [7]. The latest innovations focus on transformer-based models, such as Real-Time DETection TRansformer (RT-DETR) [8], that achieve high accuracy but are computationally demanding, limiting their applicability in resource-constrained environments.

YOLOv1 [9] was the first Convolutional Neural Network (CNN)-based system for real-time object detection. Over the years, YOLO detectors have outperformed other single-stage models [10], balancing speed and accuracy, and establishing themselves as a benchmark in the field. They are categorized into two groups: anchor-based [6], [11]–[13], and anchor-free [14], [15]. However, the generation of overlapping boxes requires Non-Maximum Suppression (NMS) post-processing, slowing down their performance. YOLOv11 [7] represents the latest available version of the YOLO family: it offers significant improvements over previous versions and is specifically

This work has been funded by the PNRR - M4C2 - Investimento 1.3, Partenariato Esteso PE00000013 - "FAIR - Future Artificial Intelligence Research" - Spoke 1 "Human-centered AI" under the NextGeneration EU program, and by the Italian Ministry of University and Research (MUR) in the framework of the FoReLab and Cross-Lab projects (Departments of Excellence).

optimized for implementation on a wide range of hardware platforms. For this reason, YOLOv11 represents a suitable compromise between model accuracy and computational efficiency, making it highly adaptable to applications on low-resource devices [7].

End-to-end detectors, characterized by simplified pipelines, saw a breakthrough with DETR, the first Transformer-based model for object recognition introduced by Carion et al. [16]. DETR eliminates hand-crafted anchors and NMS adopts a bipartite matching system for direct one-to-one predictions. However, it exhibits limitations, including slow training, high computational costs, and challenges in query optimization, making it complex and less suitable for real-time detection tasks. RT-DETR addresses computational efficiency and improves query initialization.

The purpose of this paper is to investigate the performance of modern object detection models in identifying the correct use of PPEs, focusing on their accuracy and execution speed. In particular, YOLOv11 and RT-DETR are employed on low-resource devices, such as an edge computing node, for the PPE detection task. Implementation on low-cost hardware is achieved through the use of three development frameworks, namely PyTorch, OpenVINO, and TensorRT. To the best of our knowledge, this work represents the first comparison of YOLOv11 and RT-DETR models, evaluating their performance not only in terms of predictive capability but also in efficiency on edge devices.

Overhead cranes are among the most indispensable and widespread pieces of equipment in the workplace, specifically designed to transport heavy loads and materials across large workspaces. As reported by the International Labour Organization, more than 1.3 million work-related injuries occur in the construction sector every year. The fatality rate is approximately six deaths per 100,000 workers, a figure three times higher than the average for other industrial sectors [17]. To address these challenges, we focus as a case study on the implementation of a monitoring system strategically placed beneath the overhead crane. If a hazard is detected, an alarm is triggered, and the crane's operation is immediately halted.

It is worth highlighting that the application of AI models for workplace safety, such as PPE detection, exemplifies Human-Centered AI by prioritizing worker well-being and enhancing safety measures. These systems support human operators by providing real-time risk detection and compliance monitoring, fostering safer and more efficient environments without replacing human oversight.

The rest of this work is structured as follows: Section II reviews related literature and applications in PPE detection. Section III outlines the architecture of the PPE detection system and describes the YOLOv11 network structure along with the RT-DETR network. Section IV illustrates the experimental setup, describing the data set and implementation procedures. Section V presents results highlighting accuracy, processing speed on the edge device, and power consumption. Finally, Section VI draws some concluding remarks.

II. RELATED WORK

The deployment of YOLO models on edge devices remains a key area of research, focusing on optimizing computational efficiency, reducing energy consumption, and enhancing real-time performance.

The works in [18] and [19] propose an optimized version of YOLOv8 for PPE detection, achieving a 5% increase in accuracy compared to the standard version. In [20], a private dataset of 2581 images is introduced, and several recent YOLO models are trained and their performance compared.

Recent investigations have examined diverse hardware platforms and software frameworks to improve object detection in edge environments. In [21], a YOLOv8 Pose-based monitoring system, optimized for underground mining, has demonstrated robust detection of helmets, vests, gloves, and boots under challenging conditions. The system was fine-tuned using a custom dataset to enhance accuracy, achieving $mAP_{@50}$ scores of 97% for pose estimation and 90% for PPE detection when implemented on NVIDIA Jetson Nano and Xavier NX.

In [22], a new method to improve the efficiency of object detection models for agricultural applications is introduced. A modified version of the YOLOv8n model was developed by integrating the Ghost module. Structural pruning was applied to reduce computational demands. TensorRT was employed to accelerate the models, and the research utilized the NVIDIA Jetson Nano. The integration of optimized models with the hardware facilitated real-time detection speeds exceeding 30 Frames Per Second (FPS).

The work in [23] presents a robotic system for monitoring the health of tomato plants, based on a modified version of YOLOv8n. The model, which integrates a lightweight multi-scale module in the backbone, is implemented on NVIDIA Jetson Nano and optimized with TensorRT, reaching a speed of 19 FPS.

Instead, [24] compares YOLOv7 and YOLOv8, both trained on custom datasets. YOLOv7 stands out for being more accurate than YOLOv8 in this specific task. Tests were conducted on various hardware platforms, including NVIDIA Jetson Nano, Jetson Xavier NX, and high-end GPUs such as the NVIDIA RTX 2080 Ti and GTX 1050 Ti. Specifically, YOLOv8n and YOLOv7 achieved 16 FPS and 12 FPS, respectively, on the Jetson Xavier NX.

A further comparison discussed in [25] presents a methodology for fault diagnosis in electrical equipment using infrared thermography within an edge-cloud collaborative framework. The approach incorporates the YOLOv5 model and compares its performance with YOLOv7 and YOLOv8. Findings indicate that YOLOv8 achieves superior performance. The dataset used comprises 2,261 infrared images of electrical panels acquired under various conditions, including normal, minor, moderate, and severe faults. The YOLOv5 model achieved high average accuracy in initial experiments but showed limitations under varying environmental conditions, which were addressed using an edge-cloud collaborative model updating strategy. The hardware setup included NVIDIA Jetson Nano as the edge AI platform equipped with thermal imaging modules.

III. PRELIMINARIES

A. The use case scenario

In industrial facilities, overhead cranes are used to lift and move heavy materials around a workspace. While their use simplifies operations by providing a flexible means of positioning loads, it simultaneously introduces critical safety challenges that must be addressed appropriately. The sole use of static warnings, such as warning signs, is often inadequate, as workers may ignore such instructions when operating under pressure. Therefore, active surveillance systems are necessary. As shown in Figure 1, an integrated monitoring system installed directly on the crane enables optimal observation of the area below. The crane's height facilitates the surveillance of large spaces without the need for additional cameras. Moreover, movement along the rails allows for real-time dynamic monitoring, reducing costs and eliminating ground-level equipment obstructions.

Each worker must wear appropriate PPE, such as helmets and high-visibility vests. While these precautions mitigate risks, they cannot prevent accidents if safety guidelines are not strictly followed. In high-risk areas, highlighted in red in the figure, the system promptly identifies workers failing to comply with PPE requirements. Upon detecting a violation, an alarm is triggered, and equipment may be halted if necessary to prevent fatal incidents.

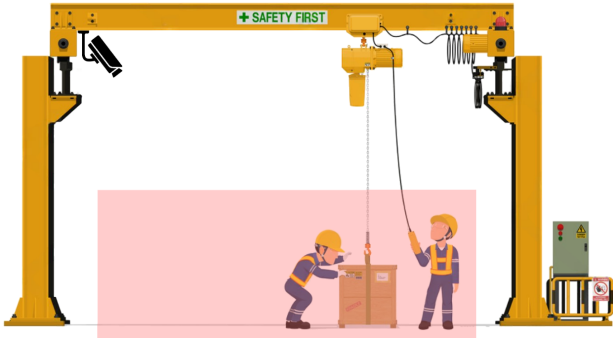


Fig. 1. A scenario featuring an overhead crane and an elevated camera. The area monitored by the PPE detection system is highlighted in red.

B. The PPE Detection System

The system employs an Intel RealSense camera and an NVIDIA Jetson module¹, strategically installed on an overhead crane. The hardware architecture is based on the NVIDIA Jetson Nano module, featuring 4GB of RAM and a cost-effective design under \$500, which serves as the execution platform. For real-time image acquisition, the Intel RealSense D435 depth camera is utilized. This device leverages infrared stereo technology to measure the distance between the camera and detected objects, providing both RGB and depth frames. The depth information is subsequently used to determine the

optimal timing for activating or deactivating the alarm in the PPE detection system. Further details can be found in [26].

The PPE detection system is fully integrated with existing infrastructures, operates directly on local hardware, and does not require recordings, as alarms are generated in real-time. This approach ensures the privacy of workers' sensitive data.

The system continuously monitors the critical zone for unprotected individuals and proximity violations. The images are obtained either from the local network or through direct connections with the cameras. Periodically, the RGB images are transmitted to a processing station, where they undergo an initial pre-processing phase, that include tasks such as scaling to ensure compatibility with the input requirements of the AI model. In addition, a depth frame is also captured to assess whether a worker is present in a hazardous zone and, if necessary, trigger appropriate safety measures. After this phase, the AI model performs inference, generating an array that indicates the probabilities and spatial positions of various object classes within predefined areas of the image. Subsequently, this information is handled by the embedded system, which generates a list of detected objects, with details about each object's position and the dimensions of its area within the image.

When an individual without appropriate PPE enters this area, an immediate alarm is activated to notify workers and supervisors. If any individual or object is detected within 2 meters of the crane or other hazardous areas, the system triggers an additional warning. Alerts are customizable and may include on-site visual signals, audible alarms, or notifications sent to a centralized safety management platform.

The software, implemented in Python, is compatible with the NVIDIA Jetson Nano. For image acquisition, using the Intel RealSense D435 depth camera, the library available in the Intel SDK² is adopted. For manipulation and processing of the captured images, the OpenCV³ library is used.

C. YOLOv11 Architecture Overview

YOLOv11 is an advanced, high-performing object detection model capable of real-time object detection, which exploits the PyTorch framework for greater speed and accuracy. The model architecture is composed of three primary components: the backbone for feature extraction, the neck for feature fusion, and the head for final object prediction [7].

The backbone component is responsible for feature extraction from input images at different scales. It consists of convolutional blocks, each structured into three main sub-blocks: Conv2D, BatchNorm2D, and the SiLU activation feature. To complement this structure, C3K2 blocks were introduced, replacing the previous C2f blocks used in YOLOv8. This modification ensures greater computational efficiency, thanks to an optimized version of the Cross-Stage Partial (CSP) model [27]. The last two backbone modules reflect a design approach focused on performance optimization. The Spatial Pyramid

¹<https://www.nvidia.com/it-it/autonomous-machines/embedded-systems/jetson-nano/product-development/>

²<https://dev.intelrealsense.com/docs/python2>

³<https://pypi.org/project/opencv-python/>

Pooling Fast (SPPF) module uses multiple max-pooling levels to efficiently extract multi-scale features [28]. At the same time, the Cross-Stage Partial with Spatial Attention (C2PSA) module applies an advanced attention mechanism [29], further increasing the accuracy of the model.

The neck component uses convolutional layers, C3K2 blocks, concatenation operations and upsampling blocks, enriched by the C2PSA mechanism. The main function of the neck is to aggregate features extracted from the backbone on different scales, and then transfer them to the next module for final processing.

Finally, the head component is dedicated to determine object classes, calculate the objectivity score, and predict bounding boxes for the identified objects.

Further details and insights into the architecture of YOLOv11 are presented in [7], [30].

D. RT-DETR Architecture Overview

RT-DETR comprises the following components: a backbone, an advanced hybrid encoder, and an integrated Transformer decoder with auxiliary prediction heads. The backbone network features from the last three stages are processed and forwarded to the encoder. The encoder refines these multi-scale features by enabling both intra-level interaction and inter-level fusion, effectively generating a sequence of rich image representations. This hybrid encoder leverages the Attention-based Intra-scale Feature Interaction (AIFI) [31] and CNN-based Cross-scale Feature Fusion (CCFF) [32] modules, which help mitigate computational bottlenecks commonly associated with Transformer models. An uncertainty-minimal query selection mechanism [33] is employed to enhance detection accuracy. By optimizing both classification and localization metrics, this mechanism provides high-quality initial object queries to the decoder. The Transformer decoder, further enhanced with auxiliary prediction heads, iteratively refines these queries to produce accurate object categories and bounding boxes. The architecture allows for flexible speed optimization by adjusting the number of decoder layers, facilitating adaptation to diverse real-time application requirements.

Further details and insights into the RT-DETR architecture can be found in [8].

Table I summarizes the main characteristics of the YOLOv11 and RT-DETR models. Specifically, for each model, the table reports the input size in pixels, the total number of parameters (million parameters, MParams), the required computational effort (GFLOPs), and the memory footprint (MB).

E. Models deployment framework

An AI model deployment framework is a set of tools and libraries designed to optimize and run deep learning models on different hardware platforms. These frameworks simplify the transition from the training phase to the inference phase.

In the context of the YOLOv11 and RT-DETR object detectors, model weights are natively provided in Pytorch

TABLE I
MAIN CHARACTERISTICS OF THE EXPERIMENTED MODELS

	Model	Input Size	MParams	GFLOPs	MBs
RT-DETR	L	640x640	32.8	108.0	63.1
	X	640x640	65.4	222.5	129.0
YOLOv11	Nano	640x640	2.6	6.5	5.2
	Small	640x640	9.4	21.5	18.2
	Medium	640x640	20.1	68.0	38.6
	Large	640x640	25.3	86.9	63.1
	XLarge	640x640	56.9	194.9	109.1

format. PyTorch⁴ is one of the most widely used frameworks for deep learning. However, using PyTorch models directly in production environments can present limitations in terms of compatibility and efficiency with specific hardware. To overcome these limitations, other frameworks like OPENVINO⁵ and TensorRT⁶ offer optimized solutions for running models on specific devices [34].

These frameworks are composed of two main elements: the Model Optimizer and the inference engine. OPENVINO's Model Optimizer converts trained models into an optimized intermediate format (IR) that is compatible with various Intel hardware architectures, including CPUs, GPUs, and VPUs. Similarly, TensorRT's Model Optimizer adapts the trained models into an optimized IR for NVIDIA devices, ensuring high performance. The Inference Engine manages the execution of the optimized models in the inference phase, effectively controlling the hardware. It is important to emphasize that optimized models do not lead to any loss of information: starting from the same input, both the unoptimized and optimized models produce identical outputs.

IV. EXPERIMENTAL SETUP

The YOLOv11 and RT-DETR models are trained using the PPE dataset introduced in [35], which was also utilized in recent research described in [26].

In detail, the dataset includes images of workers operating within different industrial environments, captured from multiple viewpoints and featuring various backgrounds, environmental conditions and distances from the camera. It is divided into three distinct groups: the first group (D_1) contains high-quality images sourced from online platforms, carefully curated to encompass a wide range of environments and scenarios; the second group (D_2) consists of images captured at a standardized distance, utilizing a green screen to facilitate the integration of artificial backgrounds and simulate complex scenarios; and the third group (D_3) represents real-world industrial settings, emphasizing workers equipped with PPE designed to address specific risks, such as gloves used during heat press operations.

Annotations for this dataset encompass six object categories: Head, Helmet, Chest, Vest, Hand and Glove. In total,

⁴<https://pytorch.org/>

⁵<https://docs.openvino.ai/2024/index.html>

⁶<https://developer.nvidia.com/tensorrt>

the dataset consists of 7283 images, which include 7223 annotated instances of the Head class, 19648 for Helmet, 19942 for Chest, 2910 for Vest, 12442 for Hand and 7062 for Glove. The distribution of instances for each class across the three image groups in the dataset is presented in Table II.

TABLE II
NUMBER OF INSTANCES PER CLASS IN EACH IMAGE GROUP OF THE DATASET.

Class	#D ₁	#D ₂	#D ₃	#D _{1,2,3}
Head	6906	136	181	7223
Helmet	19305	190	153	19648
Chest	19606	148	188	19942
Vest	2574	182	154	2910
Hand	11947	279	216	12442
Glove	6561	254	247	7062

To train the models, a high-performance server is used, equipped with two 56-core Intel(R) Xeon(R) Platinum 8480+ CPUs, 2TB of RAM, and an 80GB NVIDIA A100-SXM4 GPU. The operating system installed on the machine is Ubuntu 22.04.3 LTS, supported by CUDA 12.3 drivers.

Models are trained and tested using five-fold cross-validation. Training sessions are set to last 100 epochs, using a validation mechanism to prevent overfitting and to select the optimal weight set. The batch set size is 64. These experiments are conducted following the same protocols as those described in [26], thus allowing a direct comparison of the results.

Performance is evaluated with classical metrics [36]. In particular, the $AP_{@50}$, a specific version of the Average Precision (AP) metric, was used: an object is considered correctly detected only if it has a minimum 50% overlap with the reference object. To assess overall model performance, Mean Average Precision (mAP), defined as the mean of the AP values across all individual classes, is regarded as a key performance indicator.

The training of YOLOv11 and RT-DETR models starts with randomly generated weights, as initially explored in [26]. This method contrasts with the one described in [35], where fine-tuning is used to adapt pre-trained models for specific tasks. Starting from random weights offers full control over the learning process, albeit at the expense of longer training times. This method is particularly advantageous when the training set differs substantially from the pre-training dataset, as it helps remove biases and irrelevant features. However, this approach is only viable when a sufficient amount of high-quality data is available.

The main goal is to evaluate the accuracy, inference time, power consumption, and edge device temperature, for each trained model. Each model is subjected to three separate test runs in three distinct optimization configurations. In the first configuration, no optimization is applied and the models are run in their standard PyTorch format. In the second configuration, the models are deployed using the OpenVINO framework, while in the third configuration, the deployment uses the TensorRT framework.

The flow of the experiment is illustrated in Figure 2.

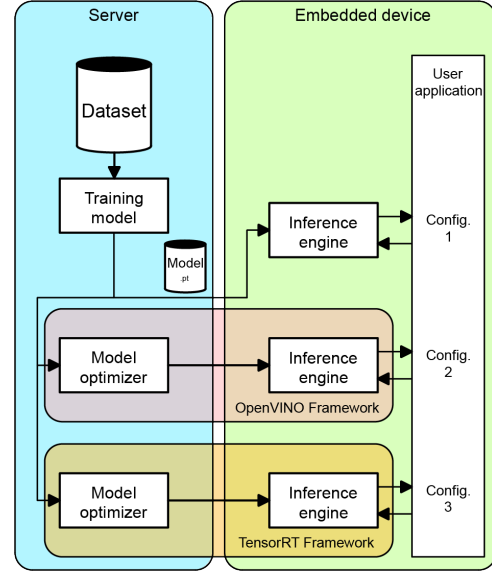


Fig. 2. Overall experiment workflow

The YOLOv11 and RT-DETR networks are trained on a high-performance server. At the end of each training session, the best weights are selected for each model. The trained models are then imported into the OpenVINO and TensorRT frameworks, which generate optimized versions executable in real time on resource-limited hardware.

V. EXPERIMENTAL ANALYSIS

A. Model accuracy and complexity

A system designed for real-time object detection must balance accuracy and frame rate. In this context, we examine the performance in terms of accuracy and frame rate of various configurations of YOLOv11 and RT-DETR. For comparison, we also include the analysis of YOLOv5 and YOLOv4 in their highest performance configurations, nano and tiny, as described in the study reported in [26].

In Tab. III, the $AP_{@50}$ values achieved by various models on the D123 dataset are presented, highlighting their performance across the six object classes considered: Head, Helmet, Chest, Vest, Hand, and Glove. Among the tested models, YOLOv11-Large, with 25.3 millions of parameters, achieves the highest overall accuracy, with $AP_{@50}$ scores of 97.5% on Head, 98.8% on Helmet, 96.0% on Chest, 90.7% on Vest, 85.5% on Hand, and 70.4% on Glove, outperforming the other models in all categories. Notably, YOLOv11-XLarge performs comparably in most classes, particularly in Helmet detection, where it matches YOLOv11-Large with an $AP_{@50}$ of 98.8%.

The RT-DETR models, available in two variants, L and X, with 32.8 million and 65.4 million parameters, respectively, achieve an average performance improvement of 0.4% and 0.7% compared to YOLOv11-Nano, though they do not outperform YOLOv11-Small. Moreover, compared to the smallest versions of YOLOv4 and YOLOv5, YOLOv11-Nano presents

comparable performance on classes with larger objects, but achieves superior accuracy on classes containing smaller objects. Specifically, for larger objects, such as Head and Helmet, YOLOv11-Nano achieves 1.4% and 1.0% higher accuracy compared to YOLOv5-Nano. In contrast, for smaller objects, such as Hand and Glove, the improvements are more significant, with increases of 5.7% and 12.4%, respectively. Meanwhile, compared to YOLOv4-Tiny, YOLOv11-Nano achieves a higher accuracy of 3.7% and 3.4% for Head and Helmet, and of 22.2% and 17.5% for Hand and Glove.

TABLE III
AVERAGE $AP_{@50}$ VALUES FOR VARIOUS MODELS ON D_{123}

Model		Head	Helmet	$AP_{@50}$		Hand	Glove
				Chest	Vest		
RT-DETR	L	96.1	97.9	93.4	84.8	81.6	64.4
	X	96.5	98.3	94.5	87.2	82.3	66.8
YOLOv11	Nano	96.6	98.2	94.9	88.1	79.9	58.7
	Small	97.1	98.6	95.4	89.4	83.0	64.7
	Medium	97.4	98.7	95.7	90.5	84.9	67.7
	Large	97.5	98.8	96.0	90.7	85.5	70.4
	XLarge	97.4	98.8	95.8	90.8	85.2	69.9
YOLOv5	Nano	95.2	97.2	92.9	84.2	74.2	46.3
YOLOv4	Tiny	92.9	94.8	92.2	83.2	57.7	41.2

Table IV presents the $AP_{@50}$ scores for each class, achieved by the YOLOv11 and RT-DETR models when evaluated exclusively on the instances of the test set belonging to dataset D_3 . These findings provide insights into the performance of the models in more practical and realistic contexts.

TABLE IV
AVERAGE $AP_{@50}$ VALUES FOR VARIOUS MODELS ON D_3

Model		Head	Helmet	$AP_{@50}$ (%)		Hand	Glove
				Chest	Vest		
RT-DETR	L	99.0	98.5	97.5	99.1	92.5	88.7
	X	98.3	99.0	97.4	99.4	88.5	93.7
YOLOv11	Nano	99.2	99.1	97.6	99.0	92.0	86.1
	Small	99.1	98.6	97.9	99.1	93.4	89.7
	Medium	99.1	98.9	97.6	99.2	92.8	90.7
	Large	99.1	98.8	97.6	99.3	92.7	90.9
	XLarge	99.0	99.1	97.0	99.3	94.3	90.5
YOLOv5	Nano	98.0	98.7	90.1	98.7	79.9	59.3
YOLOv4	Tiny	87.3	97.7	91.5	98.0	55.9	47.2

We performed the Friedman nonparametric statistical test [37], considering the $AP_{@50}$ values calculated for each class. The results show that in terms of accuracy, the YOLOv11-Large model outperforms all other networks. However, its improvement over the smaller configurations in terms of accuracy is marginal (3.5%). This observation highlights the suitability of the lighter models when applied in resource-constrained environments or applications requiring higher throughput.

B. Real-time performance of the models

In PPE sensing systems, the FPS value serves as a critical performance indicator, as a minimum FPS threshold is required to ensure the system operates effectively based on

the specific use case. To evaluate the performance of each model in terms of analyzed FPS, the models were deployed on an NVIDIA Jetson Nano module. The resulting FPS values were measured using three demonstration videos sourced from [26]. These videos simulate industrial scenarios in a laboratory environment and, in particular, include images of workers randomly removing personal protective equipment. Each model, with different optimization configurations, was tested on the integrated system, and the FPS value was measured.

Table V provides a detailed comparison of the performance metrics for YOLOv11 and RT-DETR models. The table includes FLOPs, FPS across optimizations, disk space usage, $mAP_{@50}$ values computed using cross-validation and D_3 as the test set. As anticipated, the YOLOv11-Nano model, with only 2.6 million parameters and 6.5 GFLOPs, remains the fastest. It achieves a speed of 6.6 FPS when using the Pytorch implementation and improves to 10.6 FPS with the TensorRT optimization. Progressing from the “Nano” variant to the “XLarge” variant, the number of parameters, computational demands, and storage requirements increase proportionally. The “XLarge” variant, featuring 56.9 million parameters and 194.9 GFLOPs, exhibits a significant reduction in speed, achieving only 0.7 FPS with Pytorch. However, it compensates for this limitation with superior accuracy, reaching a $mAP_{@50}$ of 96.2% on the D_3 dataset.

The RT-DETR models, in both the “L” and “X” variants, show a strong focus on accuracy, as reflected in their $mAP_{@50}$ scores, on the D_3 dataset, of 95.9% and 96.1%, respectively. However, this comes at the cost of higher computational demands and lower FPS. At FLOPs of 108.0G and 222.5G, they achieve 0.8 and 0.5 FPS with Pytorch, prioritizing accuracy over speed. When using the OpenVINO optimization, FPS performance drops further, with the “L” variant reaching 0.2 FPS and the “X” variant dropping to 0.1 FPS.

Model memory footprints show significant differences between YOLOv11 and RT-DETR across formats. YOLOv11 models demonstrate efficient storage requirements, especially in the smaller variants. For example, the “Nano” variant requires only 5.2 MB for Pytorch weights, increasing slightly to 10.2 MB and 12.5 MB for OpenVINO and TensorRT formats, respectively. As the complexity of the model increases, the memory demand also increases, with the “XLarge” variant requiring 109.1 MB for Pytorch, 251.0 MB for OpenVINO, and 221.7 MB for TensorRT. This progression reflects the trade-off between model complexity and storage efficiency, with YOLOv11 maintaining reasonable memory demands even in its largest configurations. In contrast, RT-DETR models show substantially higher storage requirements in all tested formats. The “L” variant requires 63.1 MB for Pytorch weights, doubling to 123.5 MB in the OpenVINO format, while the “X” variant requires 129.0 MB for Pytorch and as much as 251.5 MB for OpenVINO. Unlike YOLOv11, RT-DETR models do not natively support TensorRT optimizations. Overall, YOLOv11 models are better suited for storage-constrained scenarios. In contrast, RT-DETR models, while offering higher accuracy, require significantly more memory,

TABLE V
GENERAL OVERVIEW OF THE PERFORMANCE OF THE MODELS, INCLUDING FLOPs, FPS,
MAP@50 (CROSS-VALIDATION), MAP@50 (ONLY D₃ AS TEST SET) AND MEMORY OCCUPANCY ON THE DISK.

Model	MParams	FLOPs (GFlops)	FPS			mAP@50 cross-validation	on D ₃	Weights (MB)		
			Pytorch	OpenVINO	TensorRT			Pytorch	OpenVINO	TensorRT
RT-DETR	L	32.8	0.8	0.2	-*	86.4	95.9	63.1	123.5	-*
	X	65.4	0.5	0.1	-*	87.6	96.1	129.0	251.5	-*
YOLOv11	Nano	2.6	6.6	2.3	10.6	86.1	95.5	5.2	10.2	12.5
	Small	9.4	3.6	0.9	5.5	88.0	96.3	18.2	36.3	39.2
	Medium	20.1	1.7	0.3	2.3	89.2	96.4	38.6	76.8	80.7
	Large	25.3	1.2	0.2	1.9	89.4	96.4	63.1	123.0	101.4
	XLarge	56.9	0.7	-**	-**	89.6	96.2	109.1	251.0	221.7

Note: * RT-DETR models do not natively support TensorRT optimizations on NVIDIA Jetson Nano. ** Insufficient resources for real-time execution.

TABLE VI
TEMPERATURE AND POWER CONSUMPTION OF DIFFERENT MODELS ON NVIDIA JETSON NANO BOARD

Model	MParams	Pytorch ⁺	Temperature (°C)		Power Consumption (W)		
			OpenVINO ⁺⁺	TensorRT ⁺	Pytorch ⁺	OpenVINO ⁺⁺	TensorRT ⁺⁺
RT-DETR	L	32.8	72	62	-	4.9	3.4
	X	65.4	72	63	-	5.2	3.5
YOLOv11	Nano	2.6	64	62	60	3.6	2.9
	Small	9.4	70	64	70	4.3	3.2
	Medium	20.1	72	64	76	4.8	3.4
	Large	25.3	75	67	80	5.0	3.6
	XLarge	56.9	78	-	-	5.1	-

Note: ⁺ The reported temperatures and power consumption values correspond to the use of the GPU, as both Pytorch and TensorRT leverage the GPU for inference. ⁺⁺ The reported temperature and power consumption for OpenVINO refer to the CPU, as OpenVINO exclusively uses the CPU for inference on the Nvidia Jetson Nano board. All temperatures shown represent the maximum observed values during inference.

making them better suited for applications where storage constraints are less critical.

In Table VI, a comparative analysis of the temperature and power consumption for the various models of YOLOv11 and RT-DETR on the Nvidia Jetson Nano board is presented. Each result represents the average of ten repetitions to ensure consistency of the measurements. The temperature values, measured in Celsius degrees (°C), represent the maximum observed during inference. Power consumption is reported in watts (W). The RT-DETR models exhibit consistent temperatures across Pytorch and OpenVINO frameworks, with maxima of 72°C and 63°C, respectively. Power consumption for these models shows higher values for Pytorch compared to OpenVINO, with the Large variant consuming 4.9 W and 3.4 W, respectively.

For YOLOv11 models, the results indicate a wider variability in parameter configurations across different frameworks. Specifically, the Nano model achieves a minimum temperature of 60°C and a power consumption of 3.0 W when utilizing TensorRT. Notably, its lowest power consumption of 2.9 W is observed with the OpenVINO framework, though accompanied by a slightly higher temperature of 62°C. As the model complexity increases, both temperature and power consumption rise, culminating with the XLarge model, which reaches 78°C and 5.1 W under the Pytorch framework.

A notable observation is the variation in performance across frameworks. OpenVINO, utilizing CPU inference, consistently achieves lower power consumption and device temperatures

compared to PyTorch and TensorRT, which rely on GPU inference. This underscores the energy-efficient potential of CPU-based inference, although potential trade-offs in performance may be observed.

VI. CONCLUSION

In this paper, we investigated the performance of modern object detection models in identifying the correct use of PPE, focusing on their accuracy and execution speed. Specifically, we employed the YOLOv11 and RT-DETR models, trained on a real-world PPE dataset. Further, we employed three different deployment frameworks, namely PyTorch, OpenVINO, and TensorRT, to deploy the models on the NVIDIA Jetson Nano, an affordable and energy-efficient edge computing device with a power consumption of less than 10 watts. A series of experiments were conducted on a real dataset comprising six different classes to evaluate the performance of the models in terms of average precision, Frames Per Second (FPS), board temperature, and power consumption. The results show that YOLOv11n, with 2.6 million parameters, provides slightly lower average precision than more complex models. It stands out for its speed, reaching performances of 6.6 FPS with PyTorch, 2.3 FPS with OpenVINO, and 10.6 FPS with TensorRT. On the other hand, YOLOv11l and YOLOv11x with 46.5 and 86.7 million parameters, respectively, offer high average precision, especially evident in small class identification, where simpler models tend to struggle. However, they show lower throughput, with 1.2 and 0.7 FPS

on PyTorch. RT-DETR has competitive accuracy but lower real-time performance on edge devices. The evaluation also highlighted the impact of deployment frameworks on system performance. TensorRT provided the best optimization for inference speed, outperforming PyTorch and OpenVINO in all tested models. Experiments have confirmed that NVIDIA Jetson Nano, despite its low power consumption and low cost, is capable of supporting modern deep learning models for industrial applications. However, for environments that require very high-speed inference, more powerful edge devices or the use of more heavily optimized models may be required.

REFERENCES

- [1] S. Pandey, R. K. Singh, and A. Gunasekaran, "Supply chain risks in industry 4.0 environment: review and analysis framework," *Production Planning & Control*, vol. 34, no. 13, pp. 1275–1302, 2023.
- [2] S. Rasouli, Y. Alipouri, and S. Chamanzad, "Smart personal protective equipment (ppe) for construction safety: A literature review," *Safety science*, vol. 170, p. 106368, 2024.
- [3] D. Monarca, P. Rossi, R. Alemanno, F. Cossio, P. Nepa, A. Motroni, R. Gabbriellini, M. Pirozzi, C. Console, and M. Cecchini, "Autonomous vehicles management in agriculture with bluetooth low energy (ble) and passive radio frequency identification (rfid) for obstacle avoidance," *Sustainability*, vol. 14, no. 15, p. 9393, 2022.
- [4] L. Du, R. Zhang, and X. Wang, "Overview of two-stage object detection algorithms," in *Journal of Physics: Conference Series*, vol. 1544, no. 1. IOP Publishing, 2020, p. 012033.
- [5] T. Diwan, G. Anirudh, and J. V. Tembhurne, "Object detection using yolo: Challenges, architectural successors, datasets and applications," *multimedia Tools and Applications*, vol. 82, no. 6, pp. 9243–9275, 2023.
- [6] J. Redmon and A. Farhadi, "Yolo9000: better, faster, stronger," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 7263–7271.
- [7] G. Jocher and J. Qiu, "Ultralytics yolo11," <https://github.com/ultralytics/ultralytics>, 2024, version 11.0.0, licensed under AGPL-3.0.
- [8] W. Lv, S. Xu, Y. Zhao, G. Wang, J. Wei, C. Cui, Y. Du, Q. Dang, and Y. Liu, "Detrs beat yolos on real-time object detection," 2023.
- [9] J. Redmon, "You only look once: Unified, real-time object detection," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016.
- [10] S. Liu, F. Li, H. Zhang, X. Yang, X. Qi, H. Su, J. Zhu, and L. Zhang, "Dab-detr: Dynamic anchor boxes are better queries for detr," *arXiv preprint arXiv:2201.12329*, 2022.
- [11] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "Yolov4: Optimal speed and accuracy of object detection," *arXiv preprint arXiv:2004.10934*, 2020.
- [12] G. Jocher, "Ultralytics yolov5," 2020. [Online]. Available: <https://github.com/ultralytics/yolov5>
- [13] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao, "Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2023, pp. 7464–7475.
- [14] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics yolov8," 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [15] C. Li, L. Li, Y. Geng, H. Jiang, M. Cheng, B. Zhang, Z. Ke, X. Xu, and X. Chu, "Yolov6 v3. 0: A full-scale reloading," *arXiv preprint arXiv:2301.05586*, 2023.
- [16] N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with transformers," in *European conference on computer vision*. Springer, 2020, pp. 213–229.
- [17] I. L. Organization, "World statistic," https://www.ilo.org/moscow/areas-of-work/occupational-safety-and-health/WCMS_249278/lang--en/index.htm, 2024, accessed on: 12 December 2024.
- [18] B. Di, L. Xiang, Y. Daoqing, and P. Kaimin, "Mara-yolo: An efficient method for multiclass personal protective equipment detection," *IEEE Access*, 2024.
- [19] H. Zhang, C. Mu, X. Ma, X. Guo, and C. Hu, "Meag-yolo: A novel approach for the accurate detection of personal protective equipment in substations," *Applied Sciences*, vol. 14, no. 11, p. 4766, 2024.
- [20] E. Guney, H. Altin, A. E. Asci, O. U. Bayilmis, and C. Bayilmis, "Yolo-based personal protective equipment monitoring system for workplace safety," *JITSI: Jurnal Ilmiah Teknologi Sistem Informasi*, vol. 5, no. 2, pp. 77–85, 2024.
- [21] M. Imam, K. Baïna, Y. Tabii, E. Mostafa Ressami, Y. Adlaoui, I. Benzakour, F. Bourzeix, and E. Hassan Abdelwahed, "Ensuring miners' safety in underground mines through edge computing: Real-time ppe compliance analysis based on pose estimation," *IEEE Access*, vol. 12, pp. 145 721–145 739, 2024.
- [22] S. Hu, G. Tang, K. Yu, W. Chen, and Z. Wang, "Embedded yolo v8: Real-time detection of sugarcane nodes in complex natural environments by rapid structural pruning method," *Measurement*, vol. 242, p. 116291, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0263224124021766>
- [23] W. Liu, C. Bai, W. Tang, Y. Xia, and J. Kang, "A lightweight real-time recognition algorithm for tomato leaf disease based on improved yolov8," *Agronomy*, vol. 14, no. 9, 2024. [Online]. Available: <https://www.mdpi.com/2073-4395/14/9/2069>
- [24] M. Rizk and A. Chehade, "Efficient oil tank detection using deep learning: A novel dataset and deployment on edge devices," *IEEE Access*, vol. 12, pp. 170 346–170 378, 2024.
- [25] Y.-Y. Chen, S.-Y. Jhong, S.-K. Tu, Y.-H. Lin, and Y.-C. Wu, "Autonomous smart-edge fault diagnostics via edge-cloud-orchestrated collaborative computing for infrared electrical equipment images," *IEEE Sensors Journal*, vol. 24, no. 15, pp. 24 630–24 648, 2024.
- [26] G. C. Miglionico, P. Ducange, F. Marcelloni, C. Vallati, and F. Di Rienzo, "Performance evaluation of yolov5 on edge devices for personal protective equipment detection," in *2024 International Joint Conference on Neural Networks (IJCNN)*, 2024, pp. 1–8.
- [27] L. Zhou, G. Chen, L. Liu, R. Wang, and A. Knoll, "Real-time semantic segmentation in traffic scene using cross stage partial-based encoder-decoder network," *Engineering Applications of Artificial Intelligence*, vol. 126, p. 106901, 2023.
- [28] K. Xia, Z. Lv, C. Zhou, G. Gu, Z. Zhao, K. Liu, and Z. Li, "Mixed receptive fields augmented yolo with multi-path spatial pyramid pooling for steel surface defect detection," *Sensors*, vol. 23, no. 11, p. 5114, 2023.
- [29] J. Zhu, S. Chen, L. Li, and L. Ji, "Sanet: Spatial attention network with global average contrast learning for infrared small target detection," in *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2023, pp. 1–5.
- [30] R. Khanam and M. Hussain, "Yolov11: An overview of the key architectural enhancements," *arXiv preprint arXiv:2410.17725*, 2024.
- [31] Y. Feng, H. Xu, J. Jiang, H. Liu, and J. Zheng, "Icif-net: Intra-scale cross-interaction and inter-scale feature fusion network for bitemporal remote sensing images change detection," *IEEE Transactions on Geoscience and Remote Sensing*, vol. 60, pp. 1–13, 2022.
- [32] G. Cheng, Y. Si, H. Hong, X. Yao, and L. Guo, "Cross-scale feature fusion for object detection in optical remote sensing images," *IEEE Geoscience and Remote Sensing Letters*, vol. 18, no. 3, pp. 431–435, 2020.
- [33] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable detr: Deformable transformers for end-to-end object detection," *arXiv preprint arXiv:2010.04159*, 2020.
- [34] H.-Z. Zhai, J.-W. Du, Y.-H. Ai, and T.-J. Hu, "Edge deployment of deep networks for visual object detection: A review," *IEEE Sensors Journal*, pp. 1–1, 2024.
- [35] G. Gallo, F. Di Rienzo, F. Garzelli, P. Ducange, and C. Vallati, "A smart system for personal protective equipment detection in industrial environments based on deep learning at the edge," *IEEE Access*, vol. 10, pp. 110 862–110 878, 2022.
- [36] R. Padilla, S. L. Netto, and E. A. Da Silva, "A survey on performance metrics for object-detection algorithms," in *2020 international conference on systems, signals and image processing (IWSSIP)*. IEEE, 2020, pp. 237–242.
- [37] J. Derrac, S. García, D. Molina, and F. Herrera, "A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms," *Swarm and Evolutionary Computation*, vol. 1, no. 1, pp. 3–18, 2011.